

SD Basic - Modern Program Structure

Scope, local subroutines and functions, class modules and objects — writing SD BASIC the way you would write anything else.

SD BASIC has scope and it has objects. A routine can have variables of its own that nothing else can see; a class module can hold state, expose methods and properties, construct and destruct itself, and inherit from another class. None of it is a bolt-on and none of it is new — it is simply not what most MultiValue documentation talks about.

This page is that half of the language. SD Basic - Program Structure covers the traditional shape — one program, one flat variable space, `gosub` to a label, `call` to a catalogued subroutine — and everything here is an alternative to it, not an extension of it.

Everything on this page was compiled and run on SD Core for Windows W1.0-0 before it was written down. Where a listing shows an error, that error was produced, not composed. The programs are in the documentation repository — <https://github.com/dmontaine/SDCoreWindowsDocs> — as `tools\probes\p18-class-base.b`, `p18-class.b` and `p18-objects.b`.

Scope, and the one thing to get straight first

A plain `gosub` label shares every variable in the program. A **local subroutine or function** does not have to:

```
local subroutine name {(arguments)}
    ...
    return
end

local function name {(arguments)}
    ...
    return value
end
```

But a local routine is not a closed scope, and assuming it is will catch you. It can still see every variable in the main program. What it gets that a `gosub` label does not is the ability to declare names that **do not leak back** — its arguments, and anything it declares `private`.

In one program: a local subroutine takes an argument `a` and declares `private working`, and the main program then asks what it can see:

```
ZZMATH.LOCAL.SEES.MAIN=assigned(shared.probe)=1
ZZMATH.PRIVATE.LEAKED=0
ZZMATH.ARGNAME.LEAKED=0
```

a variable set in the main program	visible inside the local routine
------------------------------------	---

a name the routine declared <code>private</code>	not visible outside it
--	-------------------------------

the routine's own argument names	not visible outside it
----------------------------------	-------------------------------

So the model is **shared by default, private on request**. If you want a routine that touches nothing it was not handed, declare every working variable `private` and take everything else as an argument. The compiler helps: naming those variables in the main program afterwards raises *WARNING: WORKING is not assigned a value*, because to the main program they are different names that have never been set.

CALLING THEM: TWO DIFFERENT FORMS, AND NEITHER IS `CALL`

A local subroutine is reached with `gosub`, not `call`.

```
gosub tally(3, 4, answer)
```

`call` looks in the catalogue. Against a local subroutine it compiles cleanly and then fails at run time:

```
000002B3: Unable to load 'TALLY' object code at line 62 of
/cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/zzobj
```

A local function must be declared before it is used.

```
deffun doubled(n) local
```

Without that line the compiler has no way to know `doubled(21)` is a call, and reads it as a subscripted array:

```
57: Data item or constant not found where expected
57: Right bracket not found where expected
89: Matrix DOUBLED is not referenced in a DIM statement
WARNING: DOUBLED is not assigned a value
```

A local subroutine needs no such declaration — `gosub name(...)` is unambiguous on its own.

THE `RETURN` IS NOT OPTIONAL

A local routine that falls into its own `end` stops the program. The compiler emits `end` there as a stop, not a return, so there is **no error message and no further output** — and any object still in scope runs its destructor on the way out, which makes it look like a clean finish.

Leave the `return` out and the program prints everything up to the call, nothing after it, and exits zero. **Every one of the five local subroutines in SD's own `DEBUG` program carries an explicit `return`.**

`DEFFUN` DOES MORE THAN DECLARE

<code>deffun name(args) local</code>	a local function, defined below
<code>deffun name(args) calling "!external"</code>	an external one, under a name you choose
<code>deffun name(args) key "F5"</code>	bound to a function key
<code>deffun name(args) var.args</code>	variable argument count

`local` and `calling` are mutually exclusive — a function is either in this program or somewhere else. SD's `DEBUG` uses both forms, three lines apart.

Class modules

A class module is a program declared with `class` instead of `program`. It holds state between calls, exposes what it chooses, and is instantiated as many times as you like.

```
class name {inherits base {, base ...}} {max.args n}

private variable, ...
public variable, ...

public subroutine create.object    ... end
public subroutine destroy.object  ... end
public subroutine name{(args)}    ... end
public function name{(args)}      ... end
get name                          ... end
set name(value)                   ... end

end
```

Each member ends with its own `end`, and the module ends with one more. `private` and `public` declare member variables; `public subroutine` and `public function` declare methods; `get` and `set` declare a property.

A class is catalogued and instantiated like any other program, and **a private catalogue is enough** — nothing here needs an elevated session:

```
:basic bp zzcls
Compiling bp zzcls
0 error(s)
:catalog bp zzcls
ZZCLS added to private catalogue
```

Objects

```
object(catalogued.name {, arguments})    create an instance
objinfo(variable, 0)                     1 if the variable holds an object
objinfo(variable, 1)                     the class name
variable = ''                             release it
```

```
o = object('ZZCLS')
```

```
ZZMATH.ISOBJ=1
ZZMATH.CLASS=ZZCLS
```

-> REACHES EVERYTHING

The member operator is `->`, and it is the same operator for all four kinds of member. This is the part most worth having in front of you:

YOU WRITE

WHAT IT DOES

```
o->deposit(40)
```

calls a **public subroutine**

```
o->total(10)
```

calls a **public function** and yields its answer

```
o->label
```

reads a **public variable**

```
o->label = 'Direct'
```

writes one

```
o->owner
```

runs the **get** routine for that property

```
o->owner = 'MIXED Case Name'
```

runs the **set** routine for it

All six on one instance, with `deposit` called twice with 40 and 2. `total` multiplies by its argument; the `set owner` routine lower-cases what it is given into the public variable `label`, and `get owner` wraps it:

```
ZZMATH.TOTAL.X1=42
ZZMATH.TOTAL.X10=420
ZZMATH.LABEL.INITIAL=unnamed
ZZMATH.LABEL.AFTER=Direct
ZZMATH.OWNER=owner<mixed case name>
ZZMATH.LABEL.AFTER.SET=mixed case name
```

A property is one name with two routines behind it. `o->owner` on the right of an assignment runs `get owner`; on the left it runs `set owner(value)`. To the caller it reads exactly like a variable, which is the point — the class can validate, transform or compute without the caller knowing.

PRIVATE MEMBERS DO NOT EXIST FROM OUTSIDE

```
:crt p->balance
000003C1: Unrecognised property/method name (BALANCE). at line 92
```

Note the wording. It is not "*private*" — from outside the class the name is not a member at all. `private` is genuine encapsulation, not a convention.

The way to expose one is to write a public routine that returns it, which is what `reveal()` does in the class above.

Construction and destruction

`create.object` and `destroy.object` are hooks, not calls you make. **They run by themselves**, and this is where an object earns its keep over a set of subroutines and a common block:

```
ZZBASE.CREATE.OBJECT
ZZCLS.CREATE.OBJECT balance=0
...
ZZMATH.BEFORE.UNLOAD
ZZCLS.DESTROY.OBJECT balance=42
ZZBASE.DESTROY.OBJECT
ZZMATH.AFTER.UNLOAD
```

construction runs base first

`ZZBASE` then `ZZCLS`

destruction runs in reverse

`ZZCLS` then `ZZBASE`

releasing the variable is what fires it `o = ''`, and the two destructors ran between the lines either side of it

That is the one thing a common block cannot give you. A file left open, a lock held, a socket connected — a destructor closes it when the last reference goes, whatever route the program took to get there. SD's own client library uses exactly this: `destroy.object` in `SDCLIENT` disconnects the session if one is still up.

Inheritance

```
class derived inherits base
```

The base class's public members become the derived class's, and the derived class may override them. For a class inheriting `ZZBASE`:

```
ZZMATH.WHOAMI=ZZCLS
ZZMATH.TAG=base-tag
ZZMATH.REVEAL=base-private
```

`whoami()`

the derived version won — both classes define it

`tag`

a public **variable** of the base, reached through the derived object

`reveal()`

a base public function returning the base's own `private` member

The last row is the interesting one: **a base class's private state stays private, and reaches the outside only through whatever the base chooses to expose.** Inheritance does not flatten encapsulation.

THE RUNTIME FORM

```
inherit object
disinherit object
```

These are statements, used **inside a class**, that add or remove another *instance's* members at run time rather than at compile time. Using either outside a class module is an error — *DISINHERIT not in class*. The `inherits` clause is the one to reach for first; the statements exist for the case where what you inherit is decided while running.

objinfo and releasing

```
objinfo(v, 0)    1 if v holds an object, 0 otherwise – safe on any variable
objinfo(v, 1)    the class name; an error if v is not an object
unload.object name
```

Key 0 is the safe one. It is the only key that does not require the variable to be an object already, so it is what you test with before anything else.

Where SD itself does this

The claim that nothing uses this is wrong, and it is worth knowing what does.

gp1.bp/SDCLIENT	SD's own client library is a class module — 1,040 lines, 33 members, <code>create.object</code> and <code>destroy.object</code> both defined. Connecting to another SD server from BASIC means instantiating it
-----------------	--

gp1.bp/DEBUG	five <code>local subroutine</code> s and a <code>local function</code> , with <code>deffun ... local</code> above them
--------------	--

Choosing between the two shapes

REACH FOR	WHEN
<code>gosub label</code>	a few lines, used once, sharing the program's variables on purpose
local subroutine or function	anything with working variables of its own, or that you want to be sure touches nothing else
catalogued subroutine	logic more than one program needs
class module	state that has a lifetime — something opened, held and closed — or several instances of the same thing at once

See also

[SD Basic - Program Structure](#) · [SD Basic - Program Control](#) · [SD TCL - Programs and the Catalogue](#).